

Practical:03

AIM: Write a program to demonstrate the working of the decision tree-based ID3 algorithm.

1. Linear Regression

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load dataset
data = pd.read_csv("Salary_dataset.csv")
print("Columns in the dataset:", data.columns)

# Independent and dependent variables
X = data['YearsExperience'].values # Independent variable
Y = data['Salary'].values # Dependent Variable

# Calculate means of X and Y
X_mean = np.mean(X)
Y_mean = np.mean(Y)

# Calculate slope (b1) and intercept (b0)
numerator = np.sum((X - X_mean) * (Y - Y_mean))
denominator = np.sum((X - X_mean) ** 2)
```

$b1 = \text{numerator} / \text{denominator}$

$b0 = Y_mean - b1 * X_mean$

Prediction

$Y_pred = b1 * X + b0$

Visualization

`plt.scatter(X, Y, color="blue", label="Data Points")`

`plt.plot(X, Y_pred, color="red", label="Regression Line")`

`plt.xlabel("Years of Experience")`

`plt.ylabel("Salary")`

`plt.title("Simple Linear Regression")`

`plt.legend()`

`plt.show()`

Evaluation: Mean Squared Error (MSE)

`mse = np.mean((Y - Y_pred) ** 2)`

Print results

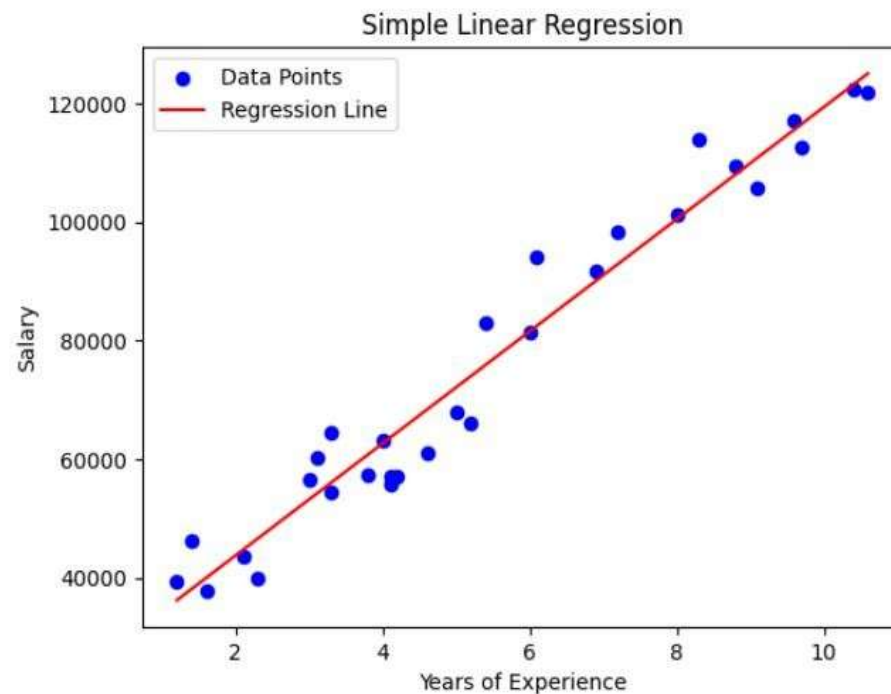
`print(f'Slope (b1): {b1}')`

`print(f'Intercept (b0): {b0}')`

`print(f'Mean Squared Error: {mse}')`

Output:

Columns in the dataset: Index(['Unnamed: 0', 'YearsExperience', 'Salary'], dtype='object')



Slope (b1): 9449.962321455076
 Intercept (b0): 24848.2039665232
 Mean Squared Error: 31270951.722280946

2. Linear Regression

Code:

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Generate synthetic binary classification data
  
```

```
X, y = make_classification(n_samples=100, n_features=2, n_classes=2, n_informative=2, n_redundant=0,
random_state=0)
```

```
# Split the data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Create and fit the model
```

```
model = LogisticRegression()
```

```
model.fit(X_train, y_train)
```

```
# Make predictions
```

```
y_pred = model.predict(X_test)
```

```
# Evaluate the model
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
class_report = classification_report(y_test, y_pred)
```

```
# Plot the results
```

```
plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', edgecolor='k', s=50)
```

```
plt.title("Logistic Regression Classification")
```

```
plt.xlabel("Feature 1")
```

```
plt.ylabel("Feature 2")
```

```
plt.show()
```

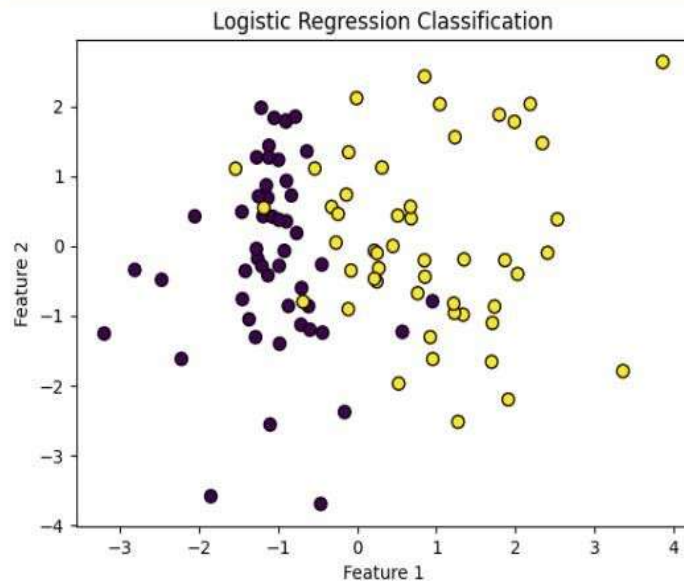
```
# Print evaluation metrics
```

```
print("Accuracy:", accuracy)
```

```
print("Confusion Matrix:\n", conf_matrix)
```

```
print("Classification Report:\n", class_report)
```

Output:



Accuracy: 1.0

Confusion Matrix:

```
[[12  0]
 [ 0  8]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	12
1	1.00	1.00	1.00	8
accuracy			1.00	20
macro avg	1.00	1.00	1.00	20
weighted avg	1.00	1.00	1.00	20

Practical: 04

AIM: Implement the naive Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering a few test data sets.

What is Naïve Baye’s Classifier?

- The Naive Bayes classifier is a family of probabilistic algorithms based on Bayes' Theorem, which is used for classification tasks.
- The algorithm assumes that the presence of a particular feature in a class is independent of the presence of any other feature, which is why it is called “naive.”

The formula is:

$$P(A|B) = \frac{P(A) P(B|A)}{P(B)}$$

Which tells us: how often A happens *given that B happens*, written **P(A|B)**,
When we know: how often B happens *given that A happens*, written **P(B|A)**
and how likely A is on its own, written **P(A)**
and how likely B is on its own, written **P(B)**

Code:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load dataset
data = pd.read_csv("train_data.csv")
X = data.iloc[:, :-1]
y = data.iloc[:, -1]
```

```
X = pd.get_dummies(X)
```

```
# Split data into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.6, random_state=42)
```

```
# Apply Naive Bayes
```

```
classifier = GaussianNB()
```

```
classifier.fit(X_train, y_train)
```

```
# Make predictions
```

```
y_pred = classifier.predict(X_test)
```

```
# Evaluate the model
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
confusion = confusion_matrix(y_test, y_pred)
```

```
report = classification_report(y_test, y_pred)
```

```
# Print evaluation metrics
```

```
print("\nAccuracy of the Naive Bayes Classifier: {:.2f}%".format(accuracy * 100))
```

```
print("\nConfusion Matrix:\n", confusion)
```

```
print("\nClassification Report:\n", report)
```

Output:

Accuracy of the Naive Bayes Classifier: 25.00%

Confusion Matrix:

```
[[0 3]
 [0 1]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.00	0.00	0.00	3
1	0.25	1.00	0.40	1
accuracy			0.25	4
macro avg	0.12	0.50	0.20	4
weighted avg	0.06	0.25	0.10	4

Practical:05

AIM: Assuming a set of documents that need to be classified, use the naive Bayesian Classifier model to perform this task.

The Naive Bayesian Classifier is commonly used for text classification tasks, such as spam detection or document categorization. Here's an implementation in Python using the Multinomial Naive Bayes model from scikit-learn to classify documents.

Steps:

1. Preprocess the text data (tokenization, removing stop words, etc.).
2. Convert text into numerical features using techniques like TF-IDF or Count Vectorization.
3. Train a Naïve Bayesian Classifier.
4. Predict and evaluate the model.

Code:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import classification_report, accuracy_score

# Sample text data
documents = [
    "I love programming in Python",
    "Python is a great programming language",
    "JavaScript is versatile and used widely",
    "I enjoy creating web applications with JavaScript",
    "Machine learning is fascinating",
    "I love studying artificial intelligence",
    "The AI revolution is here",
    "Web development with JavaScript is popular"
]

labels = ["Python", "Python", "JavaScript", "JavaScript", "AI", "AI", "AI", "JavaScript"]
```

```
# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(documents, labels, test_size=0.25, random_state=42)
```

```
# Vectorize the text data
```

```
vectorizer = TfidfVectorizer()
```

```
X_train_tfidf = vectorizer.fit_transform(X_train)
```

```
X_test_tfidf = vectorizer.transform(X_test)
```

```
# Train Naive Bayes classifier
```

```
model = MultinomialNB()
```

```
model.fit(X_train_tfidf, y_train)
```

```
# Make predictions
```

```
y_pred = model.predict(X_test_tfidf)
```

```
# Evaluate the model
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print(f'Accuracy: {accuracy:.2f}')
```

```
print("\nClassification Report:")
```

```
print(classification_report(y_test, y_pred))
```

```
# Predict new documents
```

```
new_documents = ["Python is used for data science", "Web developers use JavaScript a lot"]
```

```
new_tfidf = vectorizer.transform(new_documents)
```

```
predictions = model.predict(new_tfidf)
```

```
print("\nPredictions for new documents:")
```

```
for doc, label in zip(new_documents, predictions):
```

```
print(f"Document: {doc} -> Predicted Label: {label}")
```

Output:

Accuracy: 0.00

Classification Report:

	precision	recall	f1-score	support
AI	0.00	0.00	0.00	1.0
JavaScript	0.00	0.00	0.00	0.0
Python	0.00	0.00	0.00	1.0
accuracy			0.00	2.0
macro avg	0.00	0.00	0.00	2.0
weighted avg	0.00	0.00	0.00	2.0

Predictions for new documents:

Document: Python is used for data science -> Predicted Label: JavaScript

Document: Web developers use JavaScript a lot -> Predicted Label: JavaScript

Practical 6

AIM: Write a program to demonstrate the working of the decision tree-based ID3 algorithm.

What is ID3 algorithm?

The **ID3 (Iterative Dichotomiser 3)** algorithm is a widely used method for creating **decision trees** in machine learning. It was developed by Ross Quinlan in 1986 and is based on the concept of **information gain** from **information theory**.

Key Features of ID3:

1. Purpose:

- To construct a decision tree for classification problems.
- The resulting tree can be used to predict the class of an instance by navigating from the root to a leaf node.

2. Input:

- A dataset with features (attributes) and a target attribute (class labels).
- All attributes must be categorical (or discretized).

3. Output:

- A decision tree where each node represents a feature, branches represent feature values, and leaf nodes represent class labels.

How ID3 Works:

1. Entropy:

- It uses **entropy** to measure the level of impurity or randomness in the dataset. Lower entropy means the data is more homogeneous.

Formula:

$$\text{Entropy} = - \sum_{i=1}^n P_i \log_2 P_i$$

2. Information Gain:

- It calculates the information gain for each attribute, which is the reduction in entropy after splitting the data on that attribute.
- The attribute with the highest information gain is selected as the root node or the next decision node.

Formula:

$$\text{Information Gain} = \text{Entropy of Parent} - \sum \left(\frac{\text{Size of Subset}}{\text{Size of Parent}} \right) \times \text{Entropy of Subset}$$

3. Recursive Partitioning:

- The dataset is split into subsets based on the best attribute.
- The process is repeated recursively for each subset until:
 - All data in a subset belong to the same class, or
 - No more attributes are available, or
 - The dataset cannot be split further.

4. Stopping Criteria:

- If all examples in a subset have the same class, the node becomes a leaf.
- If no attributes remain, the node becomes a leaf with the most common class in the subset.

Example Workflow:

1. Input Dataset

Let's consider a simple dataset for classifying whether a person will play tennis based on the weather conditions.

	A	B	C	D	E
1	Outlook	Temperature	Humidity	Windy	Play
2	sunny	hot	high	FALSE	no
3	sunny	hot	high	TRUE	no
4	overcast	hot	high	FALSE	yes
5	rainy	mild	high	FALSE	yes
6	rainy	cool	normal	FALSE	yes
7	rainy	cool	normal	TRUE	no
8	overcast	cool	normal	TRUE	yes
9	sunny	mild	high	FALSE	no
10	sunny	cool	normal	FALSE	yes
11	rainy	mild	normal	FALSE	yes
12	sunny	mild	normal	TRUE	yes
13	overcast	mild	high	TRUE	yes
14	overcast	hot	normal	FALSE	yes
15	rainy	mild	high	TRUE	no

Steps:

1. Compute the entropy of the target attribute (Play Tennis).
2. Compute information gain for each feature (Outlook, Temperature, Humidity, Wind).
3. Choose the feature with the highest information gain as the root node (e.g., Outlook).
4. Split the data based on Outlook values (Sunny, Overcast, Rain).
5. Repeat recursively for each subset.

Code:

```
import math

import pandas as pd

import numpy as np

# Helper function: Calculate entropy
def calculate_entropy(data):

    elements, counts = np.unique(data, return_counts=True)

    return -sum(

        (count / len(data)) * math.log2(count / len(data))

        for count in counts if count > 0

    )

# Helper function: Calculate information gain
def calculate_information_gain(data, feature, target_attribute):

    total_entropy = calculate_entropy(data[target_attribute])

    values, counts = np.unique(data[feature], return_counts=True)

    weighted_entropy = sum(

        (counts[i] / sum(counts)) * calculate_entropy(

            data[data[feature] == values[i]][target_attribute]

        )

        for i in range(len(values))

    )

    return total_entropy - weighted_entropy

# Recursive function to build the decision tree
def build_tree(data, features, target_attribute):

    labels = data[target_attribute]
```

```
# Base cases
```

```
if len(np.unique(labels)) == 1: # Pure node
```

```
    return labels.iloc[0]
```

```
if not features: # No features left
```

```
    return labels.value_counts().idxmax()
```

```
# Find the best feature to split on
```

```
info_gains = [calculate_information_gain(data, feature, target_attribute) for feature in features]
```

```
best_feature_index = np.argmax(info_gains)
```

```
best_feature = features[best_feature_index]
```

```
# Initialize tree structure
```

```
tree = {best_feature: {}}
```

```
remaining_features = [f for f in features if f != best_feature]
```

```
# Recursively build subtrees for each value of the best feature
```

```
for value in np.unique(data[best_feature]):
```

```
    subset = data[data[best_feature] == value]
```

```
    subtree = build_tree(subset, remaining_features, target_attribute)
```

```
    tree[best_feature][value] = subtree
```

```
return tree
```

```
# Function to display the decision tree
```

```
def display_tree(tree, depth=0):
```

```
    if isinstance(tree, dict):
```

```
        for key, value in tree.items():
```

```
            print(f'| ' * depth + key)
```

```
            display_tree(value, depth + 1)
```

else:

```
print(f'| ' * depth)--> {tree}")
```

Main function to demonstrate ID3

def main():

```
print("Name: Hansha Rathod")
```

```
print("En no: 2203031050220")
```

Example dataset

```
data = {
```

```
    "Outlook": ["Sunny", "Sunny", "Overcast", "Rain", "Rain", "Rain", "Overcast", "Sunny", "Sunny", "Rain",  
"Sunny", "Overcast", "Overcast", "Rain"],
```

```
    "Temperature": ["Hot", "Hot", "Hot", "Mild", "Cool", "Cool", "Cool", "Mild", "Cool", "Mild", "Mild",  
"Mild", "Hot", "Mild"],
```

```
    "Humidity": ["High", "High", "High", "High", "Normal", "Normal", "Normal", "High", "Normal",  
"Normal", "Normal", "High", "Normal", "High"],
```

```
    "Wind": ["FALSE", "TRUE", "FALSE", "FALSE", "FALSE", "TRUE", "TRUE", "FALSE", "FALSE",  
"FALSE", "TRUE", "TRUE", "FALSE", "TRUE"],
```

```
    "PlayTennis": ["No", "No", "Yes", "Yes", "Yes", "No", "Yes", "No", "Yes", "Yes", "Yes", "Yes", "Yes",  
"No"],
```

```
}
```

```
df = pd.DataFrame(data)
```

Define features and target attribute

```
target_attribute = "PlayTennis"
```

```
features = list(df.columns[:-1])
```

Build and print the decision tree

```
decision_tree = build_tree(df, features, target_attribute)
```

```
print("\nDecision Tree:")
```

```
display_tree(decision_tree)
```

```
if __name__ == "__main__":
```

```
    main()
```

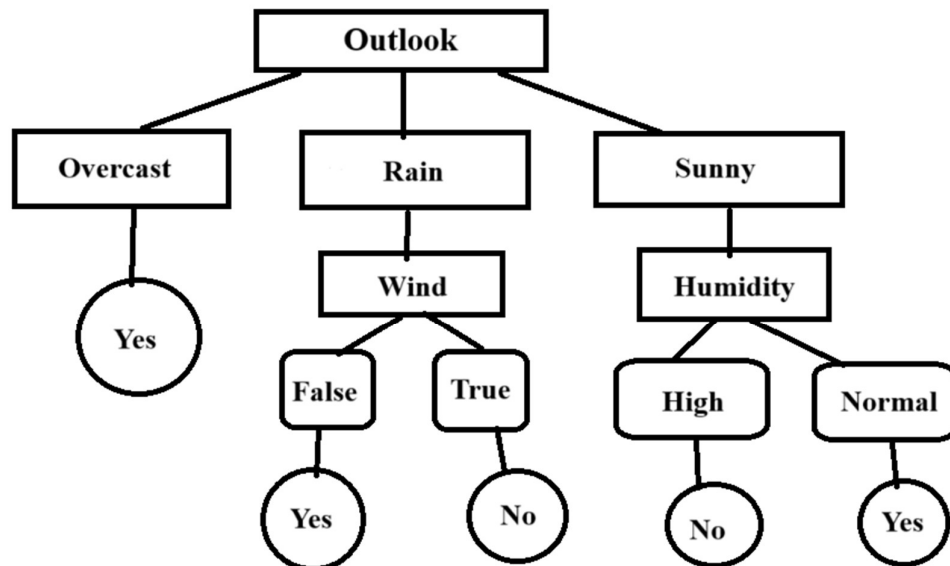

Output:

```

Name: Thura Kyaw
En no: 2203031050760

Decision Tree:
Outlook
|   Overcast
|   |   --> Yes
|   Rain
|   |   Wind
|   |   |   FALSE
|   |   |   |   --> Yes
|   |   |   TRUE
|   |   |   |   --> No
|   Sunny
|   |   Humidity
|   |   |   High
|   |   |   |   --> No
|   |   |   Normal
|   |   |   |   --> Yes

```



Decision Tree

Advantages:

- Simple and intuitive.
- Works well with small datasets and categorical data.
- Produces human-readable rules.

Disadvantages:

- Sensitive to noisy data.
- Tends to overfit if the tree grows too deep.
- Cannot handle continuous data directly without preprocessing.

Applications:

- Spam detection.
- Customer classification.
- Medical diagnosis.
- Game AI (decision-making).